

TIMER IN ATMEGA32

CSE 315

Courtesy: Muhammad Ali Nayeem, Azad Abdus Salam, Md.
Iftekharul Islam Sakib



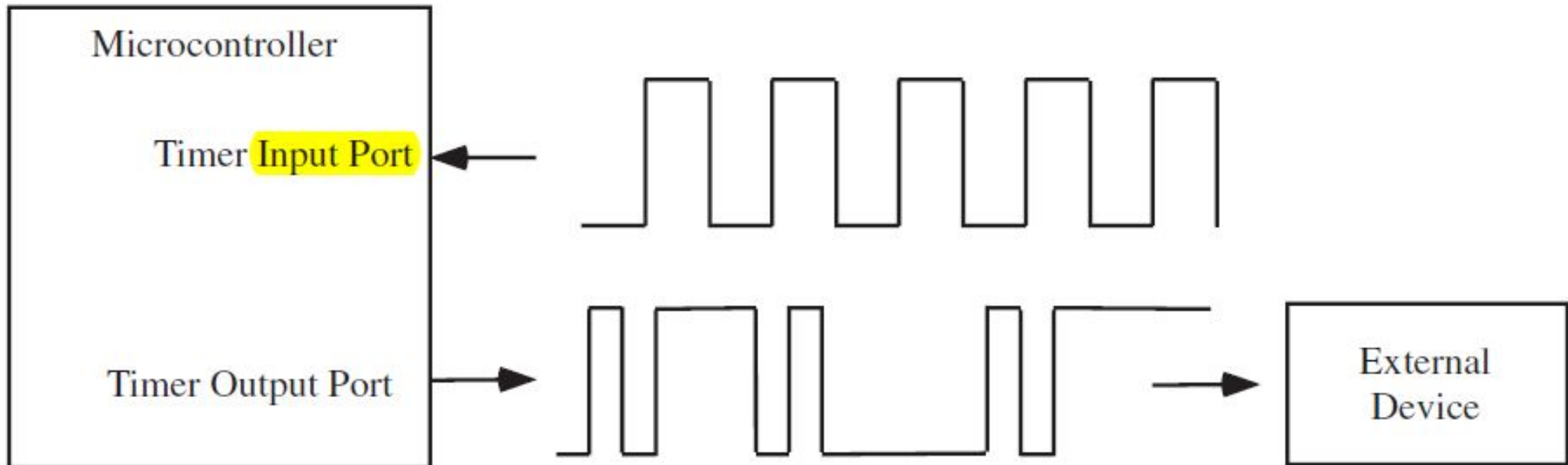
LEARNING OBJECTIVE

- What is timer?
- When, how to use timer?



TIME IS “MONEY”

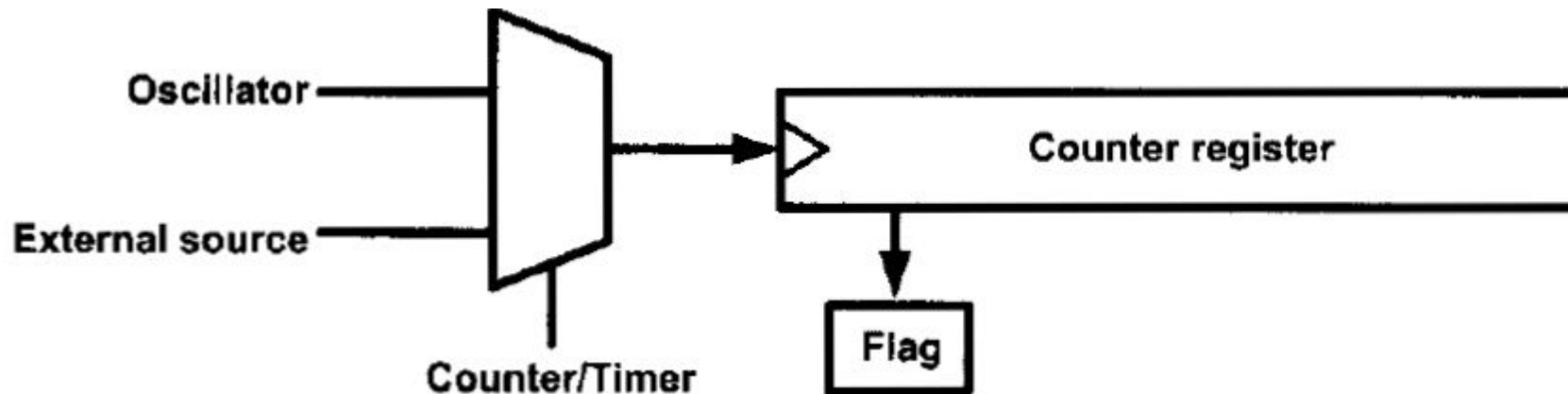
- An embedded system has to preform various **time-related tasks**
 - Recording the time when an event occurs
 - Calculating the time difference between two events
 - Performing tasks at specific or periodic time instants
 - Creating accurate time delays
 - Generating waveforms of certain shape, period or duty cycle



SOFTWARE TECHNIQUE?

- Can we use CPU to do these tasks?
 - Yes!
 - But...
 - CPU will be busy
 - Timing will not be accurate
- Need something special?
 - Hardware technique

$$1\mu S \times 2^{16} = x$$



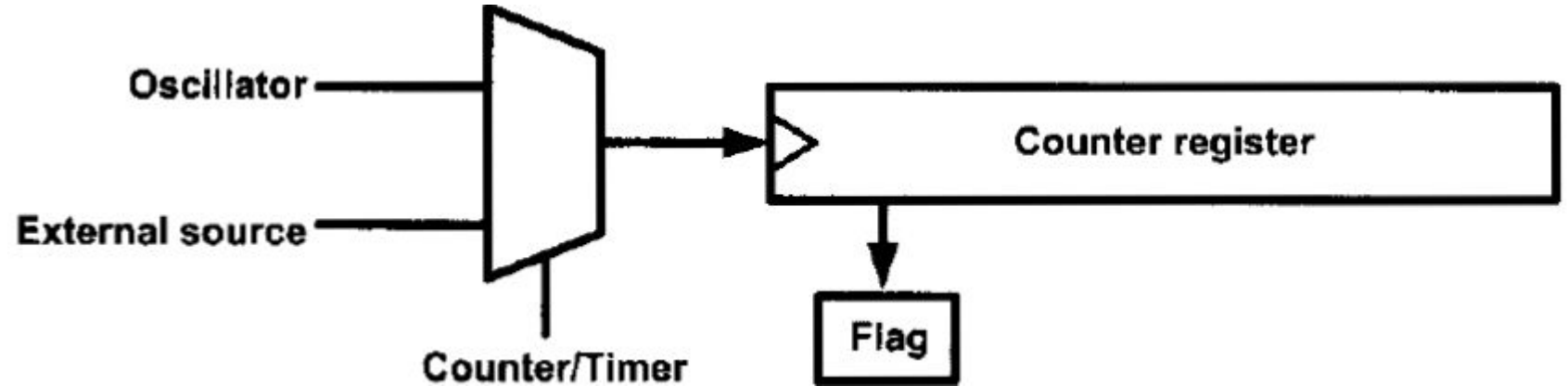
PROBLEM DELAY_MS

- If an ISR is fired during the execution of a `delay_ms()`,
 - ISR will be serviced first, then delay will continue
 - Resulting in more delay than specified
-
- For example,
 - If you call `_delay_ms(1)` and some ISR occurs that take 200us to be serviced, `_delay_ms(1)` takes 1.2ms



OVERVIEW

- ATmega32 has 3 timers
 - Timer 0, 1, 2
 - Timer 1 has the most functionalities
- Each timer is associated with
 - A counter
 - A clock signal
 - Internal
 - External



OVERVIEW

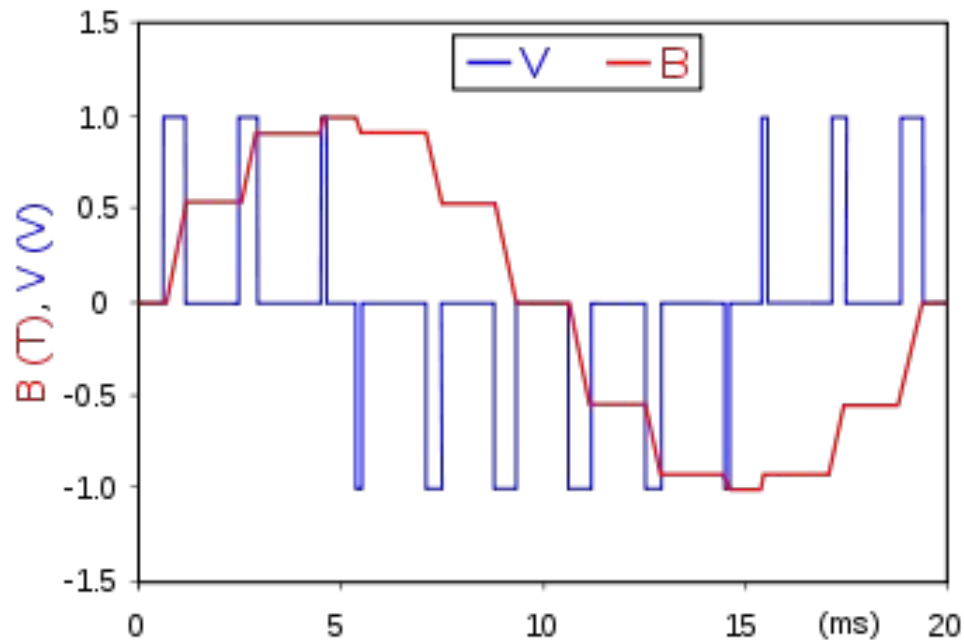
- When the **internal** system clock is used, a **prescaler** can be used to make the timer count at a slower rate
- Example:
 - Suppose the system clock rate = 1Mhz ($1\mu\text{s}$ per cycle).
 - Suppose a timer prescaler of 64 is used.
 - Then, timer will increment every $64\mu\text{s}$.



	Timer 0	Timer 1	Timer 2
Overall	- 8-bit counter - 10-bit prescaler	- 16-bit counter - 10-bit prescaler	- 8-bit counter - 10-bit prescaler
Functions	- PWM - Frequency generation - Event counter - Output compare	- PWM - Frequency generation - Event counter - Output compare: 2 channels - Input capture	- PWM - Frequency generation - Event counter - Output compare
Operation modes	- Normal mode - Clear timer on compare match - Fast PWM - Phase correct PWM	- Normal mode - Clear timer on compare match - Fast PWM - Phase correct PWM	- Normal mode - Clear timer on compare match - Fast PWM - Phase correct PWM

PWM

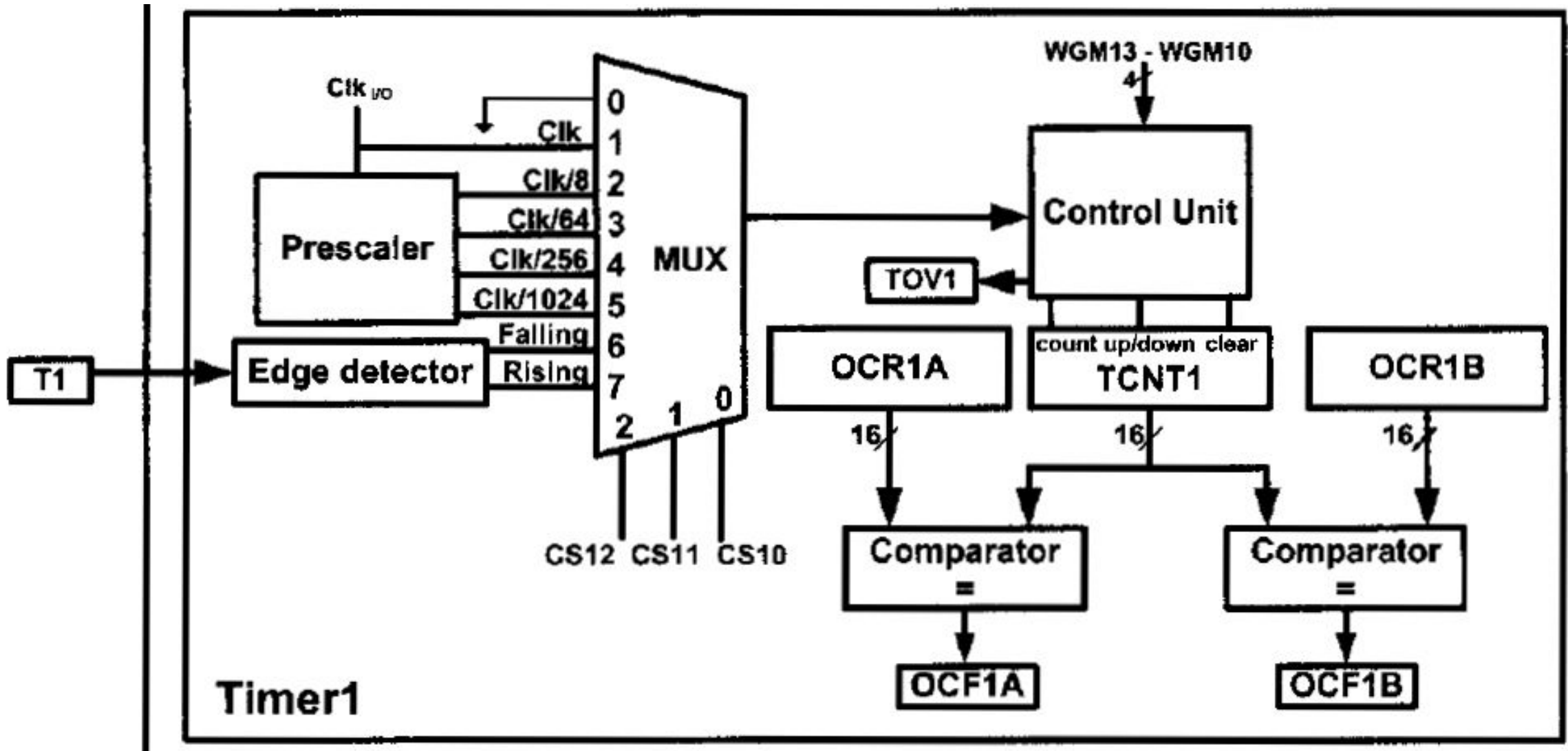
- Pulse Width Modulation
- A technique for getting analog results with digital means.



An example of PWM in an idealized inductor driven by a ■ voltage source modulated as a series of pulses, resulting in a ■ sine-like current in the inductor.



TIMER 1



TIMER 1

- 16 bit counter
 - prescaler: 8, 64, 256, 1024
- Can trigger a **timer overflow interrupt**
 - and other things.. We will learn those in the subsequent lectures
- Can trigger an **input capture interrupt** when an event occurs on the input capture pin
 - timer value is stored automatically in a register.
 - input capture pin for Timer 1 is ICP1 (D.6)

TIMER 1 – RELEVANT PINS

(XCK/T0) PB0	☐	1	40	☐	PA0 (ADC0)
(T1) PB1	☐	2	39	☐	PA1 (ADC1)
(INT2/AIN0) PB2	☐	3	38	☐	PA2 (ADC2)
(OC0/AIN1) PB3	☐	4	37	☐	PA3 (ADC3)
(SS) PB4	☐	5	36	☐	PA4 (ADC4)
(MOSI) PB5	☐	6	35	☐	PA5 (ADC5)
(MISO) PB6	☐	7	34	☐	PA6 (ADC6)
(SCK) PB7	☐	8	33	☐	PA7 (ADC7)
RESET	☐	9	32	☐	AREF
VCC	☐	10	31	☐	GND
GND	☐	11	30	☐	AVCC
XTAL2	☐	12	29	☐	PC7 (TOSC2)
XTAL1	☐	13	28	☐	PC6 (TOSC1)
(RXD) PD0	☐	14	27	☐	PC5 (TDI)
(TXD) PD1	☐	15	26	☐	PC4 (TDO)
(INT0) PD2	☐	16	25	☐	PC3 (TMS)
(INT1) PD3	☐	17	24	☐	PC2 (TCK)
(OC1B) PD4	☐	18	23	☐	PC1 (SDA)
(OC1A) PD5	☐	19	22	☐	PC0 (SCL)
(ICP1) PD6	☐	20	21	☐	PD7 (OC2)

used in this lecture →



RELATED REGISTERS – TIMER1

1. TIMER/COUNTER1

- TCNT1
- 16 bit register that stores the **current value** of the counter

2. Timer/Counter 1 Control Registers

- TCCR1A and TCCR1B
- To **configure** the operations of Timer 1.

3. Interrupt registers

- TIMSK to **enable** timer interrupts
- TIFR to monitor **status** of timer interrupts

RELATED REGISTERS – TIMER1

4. Input Capture Register

- ICR1

- to store timer value when an event occurs on input capture pin

5. Output Compare Registers

TIMER RELATED INTERRUPTS

4	TIMER2_COMP_vect	Timer/Counter2 Compare Match
5	TIMER2_OVF_vect	Timer/Counter2 Overflow
6	TIMER1_CAPT_vect	Timer/Counter1 Capture Event
7	TIMER1_COMPA_vect	Timer/Counter1 Compare Match A
8	TIMER1_COMPB_vect	Timer/Counter1 Compare Match B
9	TIMER1_OVF_vect	Timer/Counter1 Overflow
10	TIMER0_OVF_vect	Timer/Counter0 Overflow



TODAY'S MISSION

- Creating an accurate delay using timer overflow interrupt
- Measuring elapsed time between two events





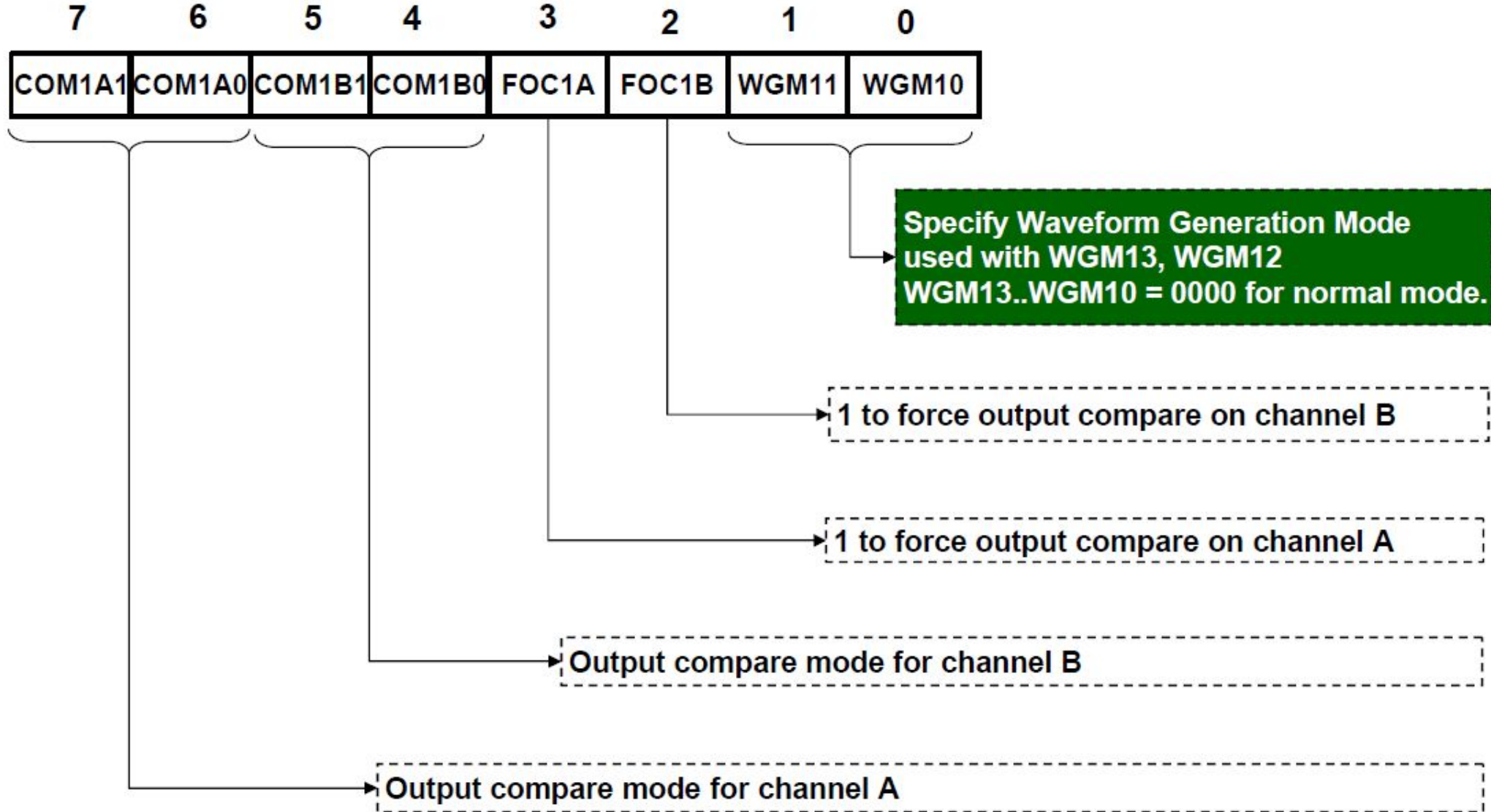
LET'S START!!!

MODES

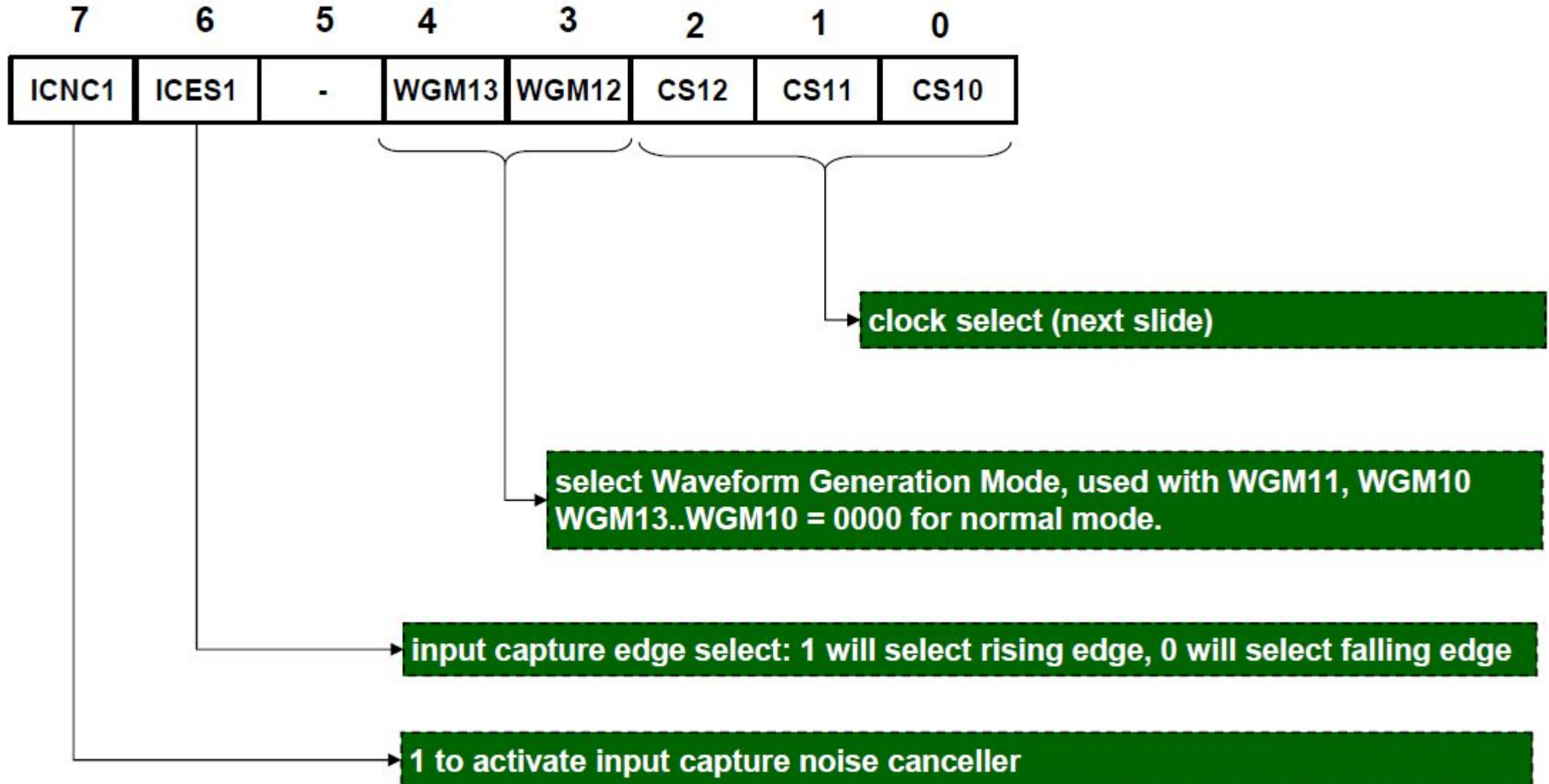
Mode	WGM13	WGM12 (CTC1)	WGM11 (PWM11)	WGM10 (PWM10)	Timer/Counter Mode of Operation	TOP	Update of OCR1x	TOV1 Flag Set on
0	0	0	0	0	Normal	0xFFFF	Immediate	MAX
1	0	0	0	1	PWM, Phase Correct, 8-bit	0x00FF	TOP	BOTTOM
2	0	0	1	0	PWM, Phase Correct, 9-bit	0x01FF	TOP	BOTTOM
3	0	0	1	1	PWM, Phase Correct, 10-bit	0x03FF	TOP	BOTTOM
4	0	1	0	0	CTC	OCR1A	Immediate	MAX
5	0	1	0	1	Fast PWM, 8-bit	0x00FF	TOP	TOP
6	0	1	1	0	Fast PWM, 9-bit	0x01FF	TOP	TOP
7	0	1	1	1	Fast PWM, 10-bit	0x03FF	TOP	TOP
8	1	0	0	0	PWM, Phase and Frequency Correct	ICR1	BOTTOM	BOTTOM
9	1	0	0	1	PWM, Phase and Frequency Correct	OCR1A	BOTTOM	BOTTOM
10	1	0	1	0	PWM, Phase Correct	ICR1	TOP	BOTTOM
11	1	0	1	1	PWM, Phase Correct	OCR1A	TOP	BOTTOM
12	1	1	0	0	CTC	ICR1	Immediate	MAX
13	1	1	0	1	Reserved	–	–	–
14	1	1	1	0	Fast PWM	ICR1	TOP	TOP
15	1	1	1	1	Fast PWM	OCR1A	TOP	TOP



TIMER/COUNTER 1 CONTROL REGISTER A (TCCR1A)



TIMER/COUNTER 1 CONTROL REGISTER B (TCCR1B)



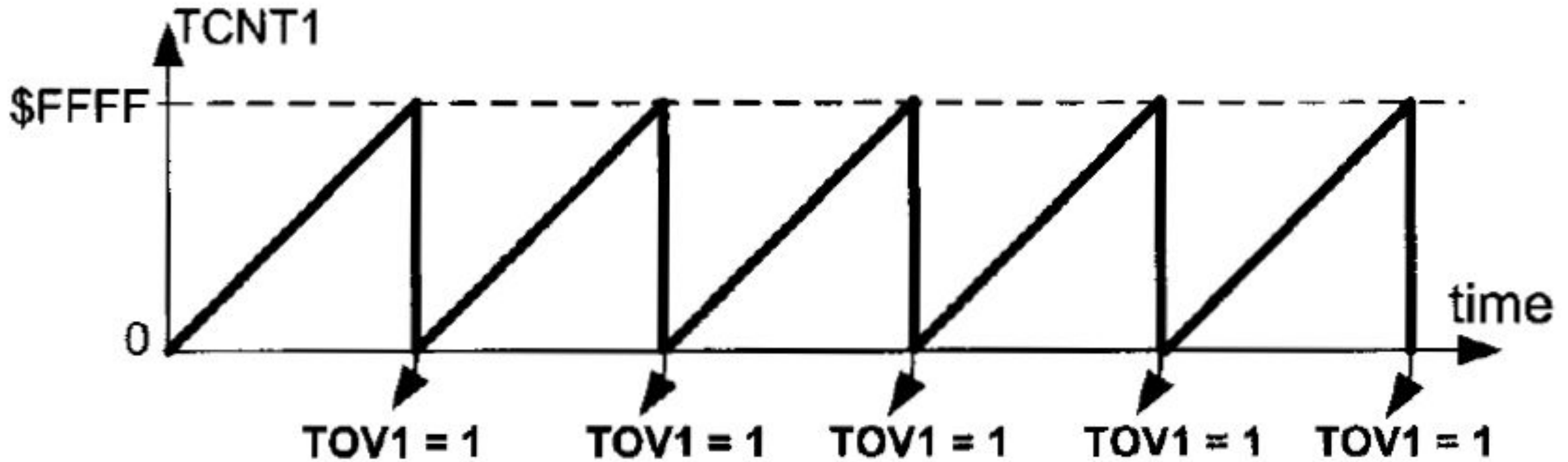
CLOCK SELECT

CS12	CS11	CS10	Description
0	0	0	No clock source (Timer/Counter stopped).
0	0	1	$\text{clk}_{\text{I/O}}/1$ (No prescaling)
0	1	0	$\text{clk}_{\text{I/O}}/8$ (From prescaler)
0	1	1	$\text{clk}_{\text{I/O}}/64$ (From prescaler)
1	0	0	$\text{clk}_{\text{I/O}}/256$ (From prescaler)
1	0	1	$\text{clk}_{\text{I/O}}/1024$ (From prescaler)
1	1	0	External clock source on T1 pin. Clock on falling edge.
1	1	1	External clock source on T1 pin. Clock on rising edge.

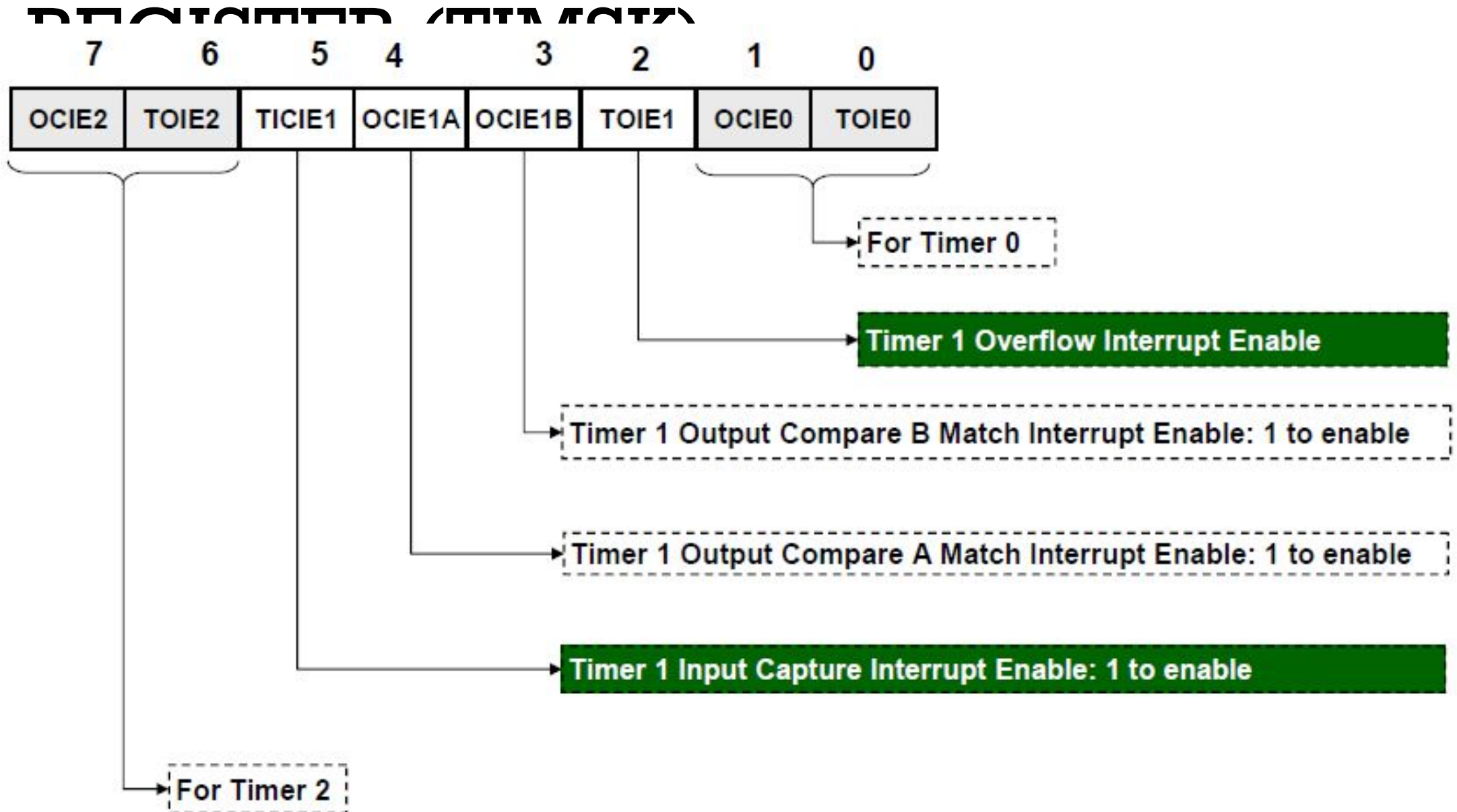
- For ATmega16, the internal clock is set by default at $\text{clk}_{\text{I/O}} = 1\text{MHz}$.
- Timer 1 can run using the internal or external clock.
- If using the internal clock, we can set Timer 1 to run at a speed that is 8, 64, 256 or 1024 times slower than the internal clock.



NORMAL MODE

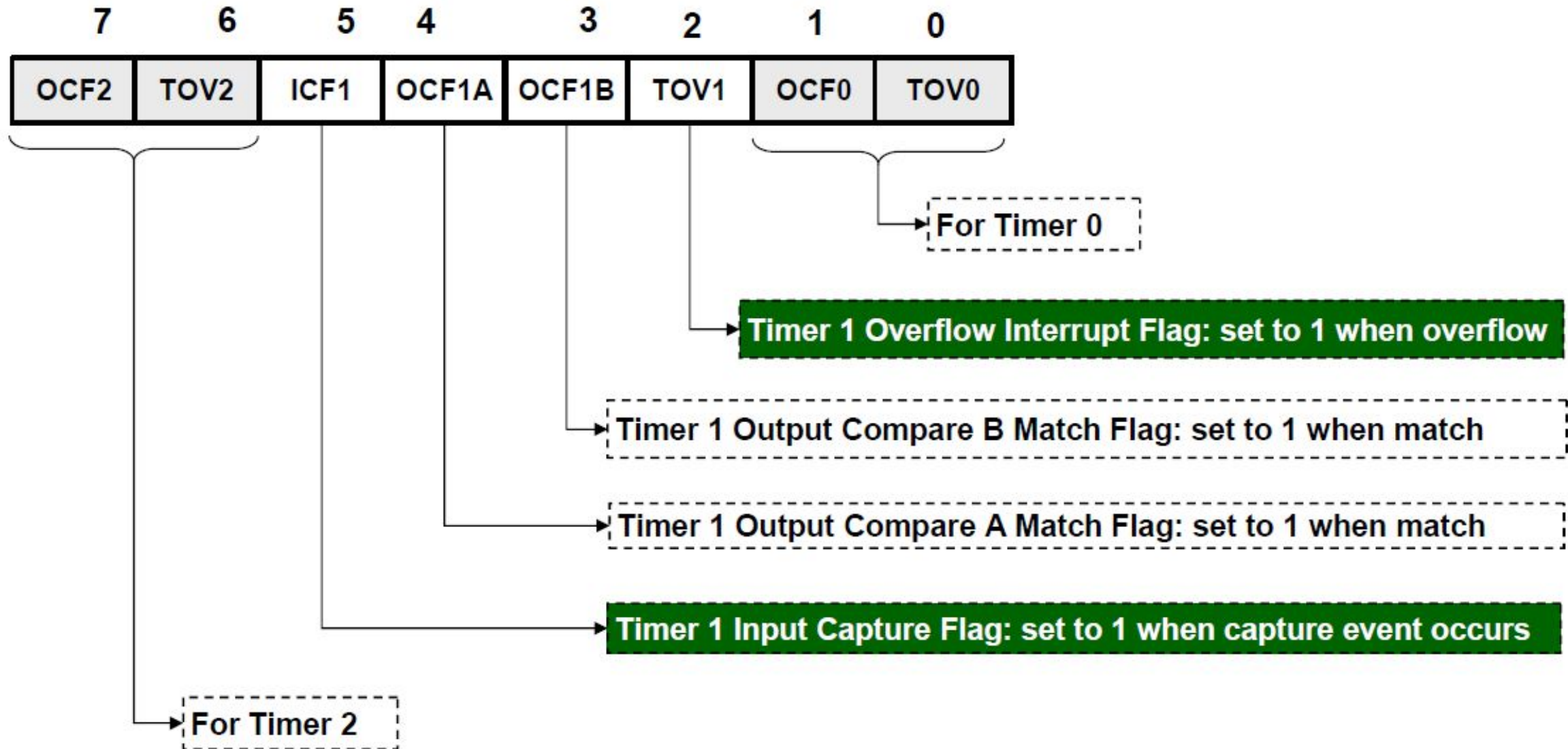


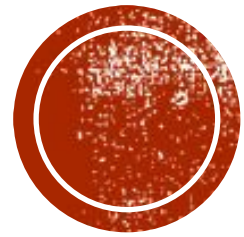
TIMER/COUNTER INTERRUPT MASK



TIMER/COUNTER INTERRUPT FLAG

DECODED (TTED)





**WRITE A C CODE TO
TOGGLE PORTB EVERY 2
SECONDS
USING TIMER 1
OVERFLOW INTERRUPT**

CREATING AN ACCURATE DELAY OF 2S

- System Default Frequency: 1Mhz
- Period: $1\mu s$
- With no prescaler TIMER 1 will increment every $1\mu s$
- TCNT1 is 16 bit
- Timer will overflow every $2^{16} \mu s$.
- How many times TIMER1 will overflow ??
- $2^{16} \mu s = 2^{16} \times 10^{-6} s$
- In 2S, total TIMER1 overflow = $\frac{2}{2^{16} \times 10^{-6}} = 30.51 \approx 31$

Bit	7	6	5	4	3	2	1	0	
	COM1A1	COM1A0	COM1B1	COM1B0	FOC1A	FOC1B	WGM11	WGM10	TCCR1A
Read/Write	R/W	R/W	R/W	R/W	W	W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Bit	7	6	5	4	3	2	1	0	
	ICNC1	ICES1	–	WGM13	WGM12	CS12	CS11	CS10	TCCR1B
Read/Write	R/W	R/W	R	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Bit	7	6	5	4	3	2	1	0	
	OCIE2	TOIE2	TICIE1	OCIE1A	OCIE1B	TOIE1	OCIE0	TOIE0	TIMSK
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

```
#include <avr/io.h>
#include <avr/interrupt.h>

volatile int overflowCount;
ISR(TIMER1_OVF_vect)
{
    overflowCount++;
    if(overflowCount==31)
    {
        PORTB = ~PORTB;
        overflowCount = 0;
    }
}
```

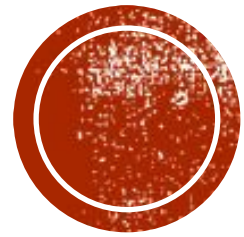
```
int main(void)
{
    overflowCount = 0;
    DDRB = 0xFF;
    PORTB = 0xFF;
    //configure timer
    TCCR1A = 0b00000000; // normal mode
    TCCR1B = 0b00000001; // no prescaler, internal
    clock

    TIMSK = 0b00000100; //Enable Overflow Interrupt
    sei(); //Global Interrupt Enable

    while(1);
}
```

APPENDIX

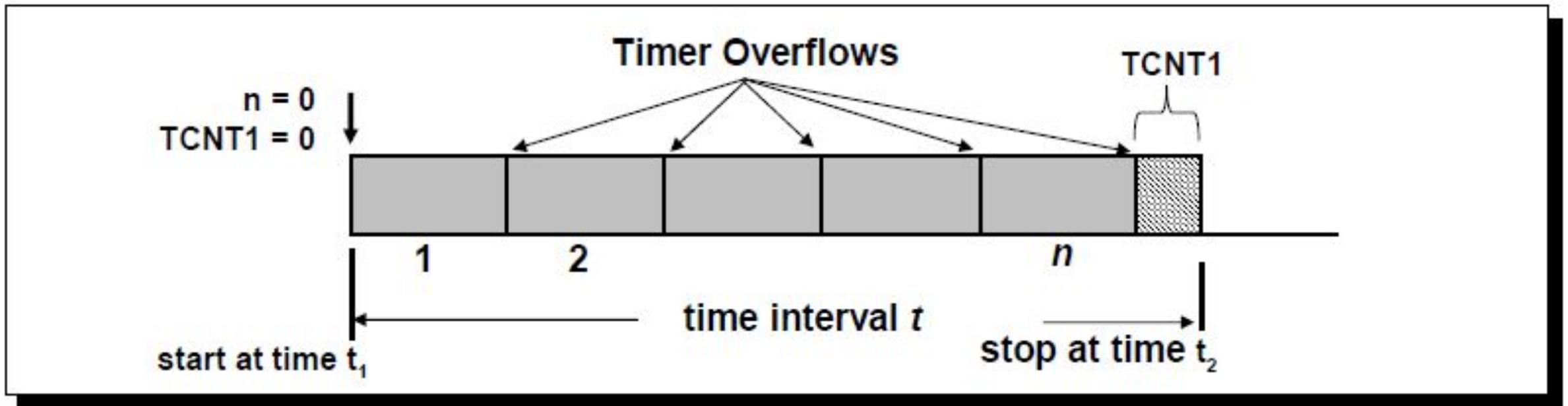
- Why volatile variable?
 - <http://www.avrfreaks.net/forum/whats-volatile-variable>



**MEASURE THE TIME
FOR A DEMO C
CODE TO EXECUTE**

MEASURING ELAPSED TIME

$$t = n \times 65536 + \text{TCNT1} \quad (\mu\text{s}) \quad \text{When no prescalling}$$



MEASURING ELAPSED TIME

- 16 Bits
- 2^{16} Counts to overflow
- Find times between increments (Period)
- For no prescaler, frequency, $F = 10^6 / 1 = 10^6$
- Period, $P = 1 / F = 10^{-6} = 1\mu s$

- Number of overflows = n
- Total count, $C = 2^{16} \times n + \text{TCNT1}$
- Total time = $C \times P$



Bit	7	6	5	4	3	2	1	0	
	COM1A1	COM1A0	COM1B1	COM1B0	FOC1A	FOC1B	WGM11	WGM10	TCCR1A
Read/Write	R/W	R/W	R/W	R/W	W	W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Bit	7	6	5	4	3	2	1	0	
	ICNC1	ICES1	–	WGM13	WGM12	CS12	CS11	CS10	TCCR1B
Read/Write	R/W	R/W	R	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Bit	7	6	5	4	3	2	1	0	
	OCIE2	TOIE2	TICIE1	OCIE1A	OCIE1B	TOIE1	OCIE0	TOIE0	TIMSK
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

```
#include <avr/io.h>
#include <avr/interrupt.h>
#include <inttypes.h>

volatile uint32_t n;
ISR(TIMER1_OVF_vect){    // handler for Timer1 overflow interrupt
    n++;                // increment overflow count
}

int main(void) {
    int i, j;
    uint32_t elapse_time;

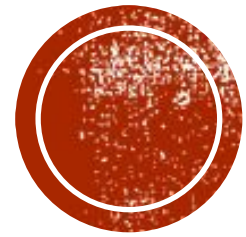
    TCCR1A = 0b00000000; // normal mode
    TCCR1B = 0b00000001; // no prescaler, internal clock
    TIMSK  = 0b00000100; // enable Timer 1 overflow interrupt

    n = 0;                // reset n
    TCNT1 = 0;            // reset Timer 1
    sei();                // enable interrupt subsystem globally

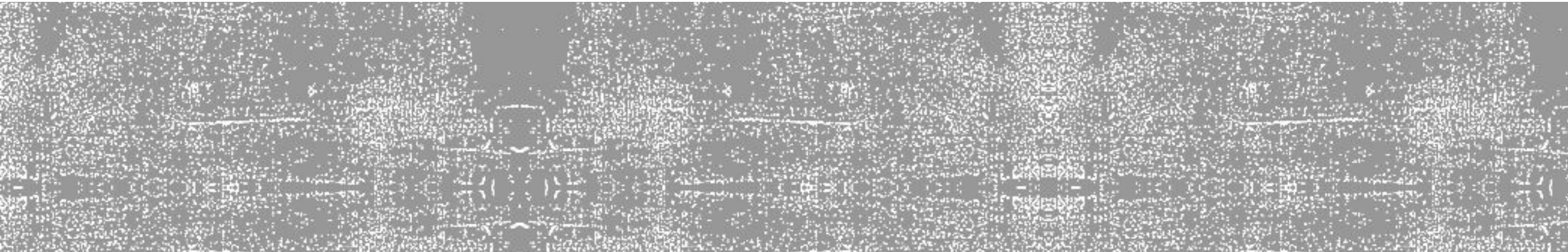
    // ----- start code -----
    for (i = 0; i < 100; i++)
        for (j = 0; j < 1000; j++){;}
    // ----- end code -----

    elapse_time = n * 65536 + (uint32_t) TCNT1; // In micro seconds
    cli();                // disable interrupt subsystem globally
    return 0;
}
```





MEASURE THE PERIOD OF A SQUARE WAVE

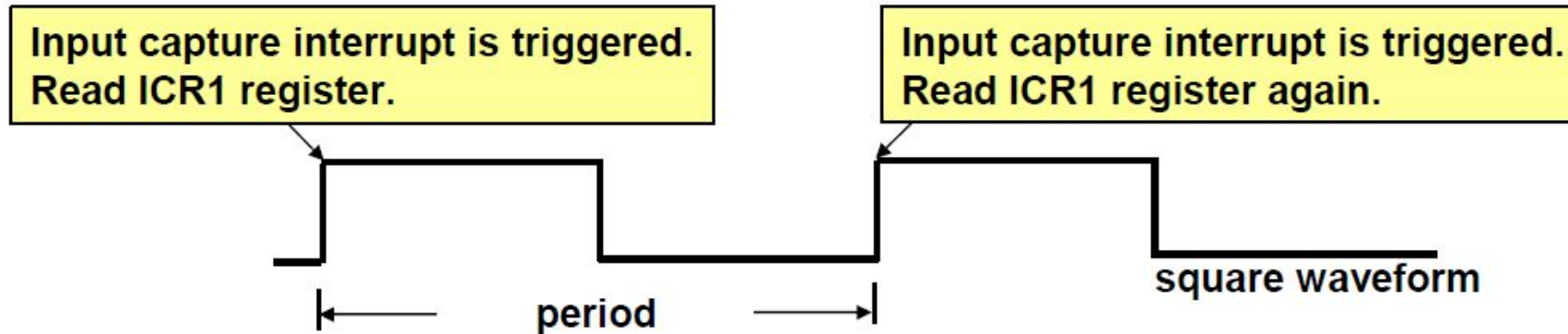


MEASURE PERIOD

■ Analysis:

- ❑ The period of a square wave = the time difference between two consecutive rising edges.
- ❑ Connect the square wave to input capture pin of Timer 1. →
- ❑ Configure input capture module to trigger on a rising edge.

PDIP			
(XCK/T0) PB0	1	40	PA0 (ADC0)
(T1) PB1	2	39	PA1 (ADC1)
(INT2/AIN0) PB2	3	38	PA2 (ADC2)
(OC0/AIN1) PB3	4	37	PA3 (ADC3)
($\overline{SS}$) PB4	5	36	PA4 (ADC4)
(MOSI) PB5	6	35	PA5 (ADC5)
(MISO) PB6	7	34	PA6 (ADC6)
(SCK) PB7	8	33	PA7 (ADC7)
RESET	9	32	AREF
VCC	10	31	GND
GND	11	30	AVCC
XTAL2	12	29	PC7 (TOSC2)
XTAL1	13	28	PC6 (TOSC1)
(RXD) PD0	14	27	PC5 (TDI)
(TXD) PD1	15	26	PC4 (TDO)
(INT0) PD2	16	25	PC3 (TMS)
(INT1) PD3	17	24	PC2 (TCK)
(OC1B) PD4	18	23	PC1 (SDA)
(OC1A) PD5	19	22	PC0 (SCL)
(ICP1) PD6	20	21	PD7 (OC2)



When a capture is triggered, the 16-bit value of the counter (TCNT1) is written to the *Input Capture Register (ICR1)*.




```
#include <avr/io.h>
#include <avr/interrupt.h>
#include <inttypes.h>
```

```
volatile uint16_t period;
```

```
ISR(TIMER1_CAPT_vect) { // handler for Timer1 input capture interrupt
    period = ICR1;        // period = value of Timer 1 stored in ICR1
    TCNT1 = 0;           // reset Timer 1
    PORTB = ~(period >> 8); // display top 8-bit of period on PORT B
}
```

```
int main(void) {
    DDRB = 0xFF;          // set port B for output

    TCCR1A = 0b00000000; // normal mode
    TCCR1B = 0b11000001; // no prescaler, rising edge, noise canceller
    TIMSK = 0b00100000; // enable Timer 1 input capture interrupt
    sei();               // enable interrupt subsystem globally
    while (1) {;}        // infinite loop
    return 0;
}
```



MEASURE THE PERIOD OF A SQUARE WAVE

- Exercise:
- No assumption on frequency
- Period of input signal is greater than $2^{16}\mu s$.
- Need to consider overflow count.
- Left as an exercise to the reader.

